MINOR PROJECT | CNN + STREAMLIT

CIFAR-10 Image Classification

A convolutional neural network with an interactive image classifier

STUDENT
Chaitanya Rao

BATCH
To be provided

DATE
01 November 2026

PROJECT OVERVIEW

This individual project applies a CNN to the course-provided CIFAR-10 images and presents the model in a simple Streamlit application. The workflow covers image and label preparation, training, held-out evaluation, sample predictions, and an upload-based demo.

DATASET
CIFAR-10
50,000 labeled RGB images

MODEL
Compact CNN
10 object categories

APPLICATION
Streamlit
Upload and classify an image

Objective

Build and evaluate a CNN that recognizes one of ten CIFAR-10 classes, then make its prediction available through a clear upload-and-result interface. The report records the real training run and explains the limits of the supplied unlabeled competition test archive.

Submission files: one Python project file and this PDF report.

Prepared from the assignment brief and the dataset supplied in the project folder.

01 / DATA

Dataset and preprocessing

CIFAR-10 contains small color images across ten balanced object classes.

The supplied labeled archive contains 50,000 PNG images, each 32 x 32 pixels with three RGB channels. The label file assigns 5,000 images to each class. Images are matched to labels by numeric file ID.

50,000
labeled training images

10
balanced object classes

32 x 32
RGB pixels per image

Class names

airplane | automobile | bird | cat | deer
dog | frog | horse | ship | truck

Labeled data split

Partition	Images	Use
Training pool	40,000	10,000 used (1,000/class)
Validation	5,000	Tune and track learning
Held-out test	5,000	Final labeled evaluation

Preprocessing and training workflow

1. Decode PNG images and convert to RGB. **2.** Keep image values as bytes until the model rescales pixels to 0-1. **3.** Make stratified 80/10/10 splits from the 50,000 labeled examples. **4.** Use a balanced 10,000-image training subset for a practical CPU run. Random horizontal flips and small translations augment training images.

Sample predictions

The test examples below show both correct (green) and incorrect (red) predictions from the held-out set.



Test-data note: the supplied 300,000-image competition test archive has no ground-truth labels. The 5,000-image labeled holdout is used for the accuracy score; the sample-submission file is not treated as ground truth.

02 / MODEL

CNN architecture

The model uses convolution and pooling to learn image features before a 10-class softmax output.

Four Conv2D layers use 32 and 64 filters, followed by max pooling and dropout in two blocks. Global average pooling feeds a 96-unit dense layer and the final 10-unit output. The model has 72,778 trainable parameters.

```
Model: "cifar10_compact_cnn"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 image (InputLayer)          [(None, 32, 32, 3)]       0

 data_augmentation (Sequent  (None, 32, 32, 3)         0
 ial)

 rescaling (Rescaling)       (None, 32, 32, 3)         0

 conv2d (Conv2D)             (None, 32, 32, 32)        896

 conv2d_1 (Conv2D)           (None, 32, 32, 32)        9248

 max_pooling2d (MaxPooling2  (None, 16, 16, 32)        0
 D)

 dropout (Dropout)           (None, 16, 16, 32)        0

 conv2d_2 (Conv2D)           (None, 16, 16, 64)        18496

 conv2d_3 (Conv2D)           (None, 16, 16, 64)        36928

 max_pooling2d_1 (MaxPoolin  (None, 8, 8, 64)          0
 g2D)

 dropout_1 (Dropout)         (None, 8, 8, 64)          0

 global_average_pooling2d (  (None, 64)                0
 GlobalAveragePooling2D)

 dense (Dense)               (None, 96)                6240

 dropout_2 (Dropout)         (None, 96)                0

 class_probabilities (Dense  (None, 10)                970
 )

=================================================================
Total params: 72778 (284.29 KB)
Trainable params: 72778 (284.29 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

Model summary generated by TensorFlow/Keras during the training run.

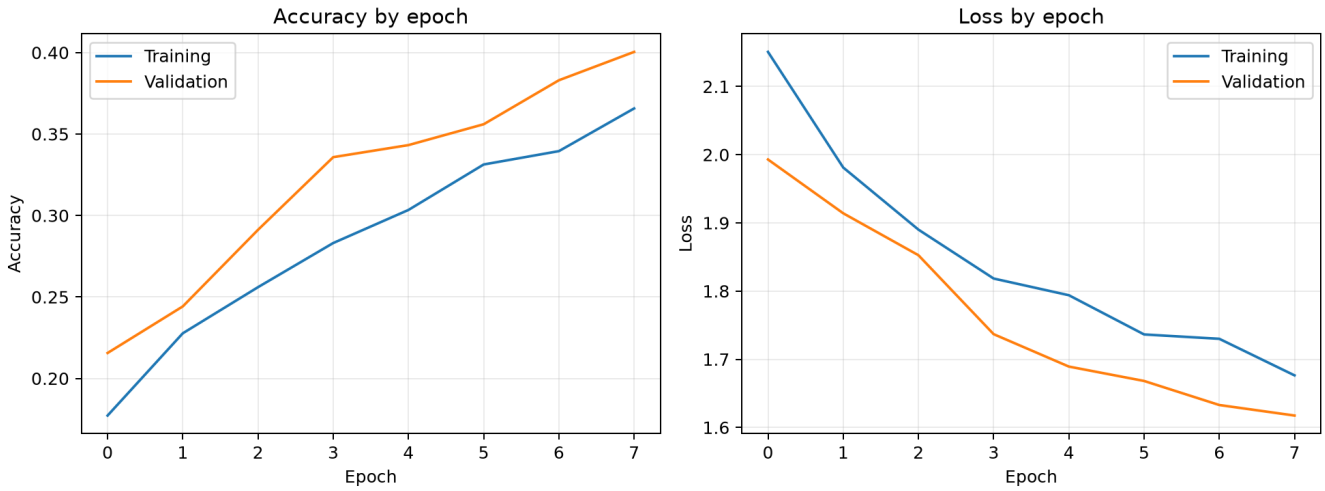
03 / RESULTS

Training and evaluation

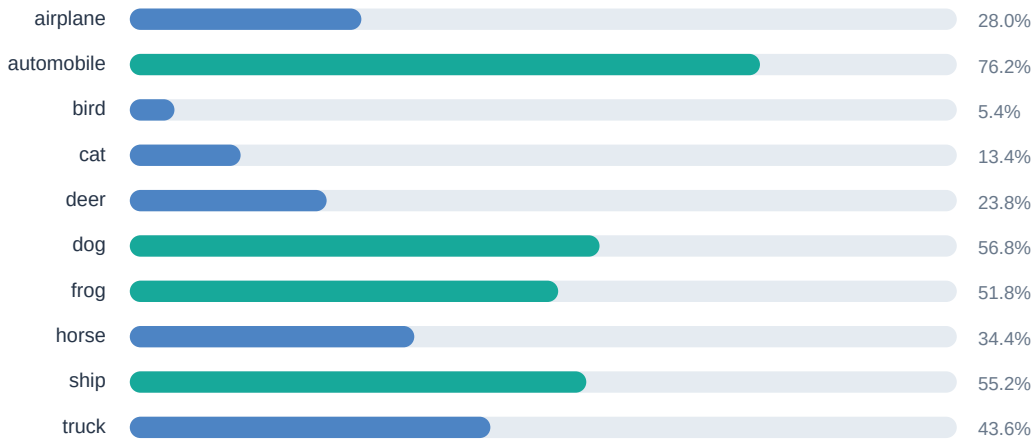
All metrics below are from the saved eight-epoch run on a fixed stratified split.

36.6%	40.0%	38.9%	1.637
training accuracy	validation accuracy	held-out test accuracy	held-out test loss

Accuracy and loss by epoch



Per-class held-out accuracy



The model classified 1,943 of 5,000 held-out images correctly (38.9%). Validation accuracy rose from 21.6% to 40.0% while validation loss fell from 1.993 to 1.617. It learned more during training, but performance remains moderate: the short run used 10,000 of the available 40,000-image training pool. Automobile was strongest (76.2%); bird was weakest (5.4%).

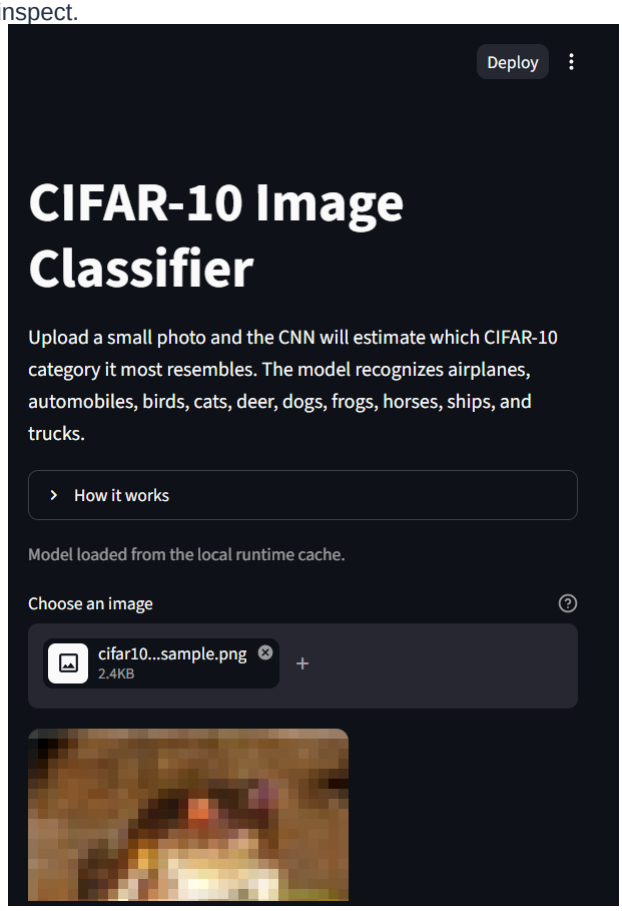
The final training accuracy was 36.6% and the final training loss was 1.676. Validation values were 40.0% accuracy and 1.617 loss. The validation curves are higher/lower than training in part because training uses dropout and data augmentation.

04 / APPLICATION

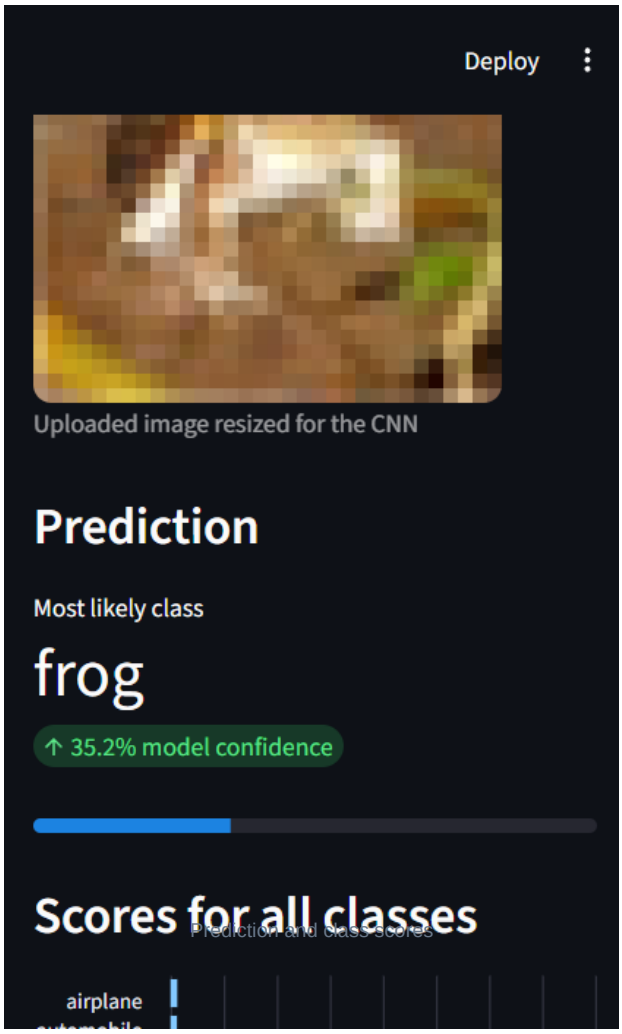
Image upload and prediction

The app accepts an image, preprocesses it, and displays the predicted class with all class scores.

The user uploads a PNG, JPG, JPEG, or WebP file. The app handles invalid image data, converts valid images to RGB, resizes them to 32 x 32 pixels, and returns the highest-scoring CIFAR-10 class. A class-score chart makes the model output easy to inspect.



Upload view



App requirements covered

Clear title and instructions; image-file input; RGB conversion and resizing; top prediction and confidence; score chart across all ten classes; and invalid-file handling with a helpful message.

Conclusion and next steps

This project completes the CNN workflow and connects it to a usable Streamlit interface. The experiment shows that accuracy improves as loss falls, while the modest test score and weak bird results point to under-training. A next run can use the full training pool, more epochs, and per-class error analysis. Batch normalization, learning-rate tuning, and a confusion matrix would also be useful improvements.

References: CIFAR-10 files and labels supplied with the course project; TensorFlow/Keras used for model training; Streamlit used for the application interface.

The 300,000-image competition test set is unlabeled; no accuracy is claimed for those images.